Al-Based Phishing Website Detection using Hybrid Feature Learning

A Project Report

Submitted by:

Piyush Kumar Pradhan(2241016408)

Raja Kishor Nayak(2241014110)

Tanmaya Mangaraj(2241013095)

Aditya Pal (2241016031)

S Ramananda Sagar(2241019007)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHONOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty of Engineering and Technology, Institute of Technical Education and Research

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

Bhubaneswar, Odisha, India

(June 2026)

CERTIFICATE

This is to certify that the project report titled —**Al-Based Phishing Website Detection using Hybrid Feature Learning**‖ being submitted by Piyush Kumar Pradhan, Raja Kishor Nayak, Tanmaya Mangaraj, Aditya Pal, S Ramananda Sagar to the Institute of Technical Education and Research, Siksha 'O' Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering is a record of original confide work carried out by them under my/our supervision and guidance. The project work, in my/our opinion, has reached the requisite standard fulfilling the requirements for the degree of Bachelor of Technology.

The results contained in this project work have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Mr. Subhasish Mohanty

Department of Computer Science and Engineering

Faculty of Engineering and Technology;
Institute of Technical Education and Research;
Siksha 'O' Anusandhan (Deemed to be) University

ACKNOWLEDGEMENT

Phishing Website Detection using Hybrid Feature Learning based on Artificial Intelligence.

Lastly, I would like to acknowledge my all faculty members and Our team members whose support and contributions in some way or another have made this research possible. We would like to express our sincere gratitude to Mr. Subhasish Mohanty, our project supervisor for his continuous guidance, advice and motivation throughout our research process. His expert guidance and advice regarding various fields of Artificial Intelligence and Machine Learning helped us a lot in carrying out this research.

We are also grateful to the Department of Computer Science & Engineering and Faculty of Engineering & Technology at ITER, Siksha 'O' Anusandhan (Deemed to be) University, Bhubaneswar for providing us with infrastructure and facilities which helped us carry out our research successfully.

We are grateful to those people who have contributed to the open source platforms like Google Colab, Scikit Learn, Tensorflow/Keras, XGBoost, SHAP, and Kaggle for making their useful contributions in terms of tools and data sets for developing our proposed system.

Signature of Students

1.

2.

3.

4.

5.

Place: Bhubaneswar,Odisha

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where other‘s ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Signature of Students with Registration Numbers

1. 2241016408
2. 2241014110
3. 2241013095
4. 2241016031
5. 2241019007

Date:

REPORT APPROVAL

This project report titled **"Al-Based Phishing Website Detection using Hybrid Feature Learning"** submitted by Piyush Kumar Pradhan, Raja Kishor Nayak, Tanmaya Mangaraj, Aditya Pal, S Ramananda Sagar is approved for the degree of *Bachelor of Technology in Computer Science and Engineering*.

Examiner(s)

_____

_____

_____

Supervisor

_____

Project Coordinator

_____

PREFACE

Phishing websites are bad news. They are one of the threats on the internet today. Bad people make websites that look like real ones we trust. They do this to steal our information like usernames and passwords.To resolve this, this project engineers a hybrid Feature Learning ML Model pipeline using 8 Basic Model. Moreover, the framework does not rely on any external service provider. Using data from various datasets sourced from Open Source, we were able to create a balanced dataset of 10,000 samples (5,000 phishing, 5,000 legitimate). The highest achieved performance is produced by the Stacking Ensemble with 90.95% (ROC-AUC: 0.9608) while Gradient Boosting gives 90.65% and Voting Ensemble gives 90.55%. The SHAP analysis shows that URLs length, path length, and file name length are the most discriminative features The proposed framework displays that phishing can be detected with an accuracy of almost about 91% using only the URL-Features.

INDIVIDUAL CONTRIBUTIONS

Piyush Kumar Pradhan	Studied phishing detection methods, handled datasets,tested model, and documented results.
Raja Kishor Nayak	Learned boosting models, hyperparameter tuning, ROC-AUC evaluation, and model optimization.
Tanmaya Mangaraj	Learned ANN design, overfitting prevention, SHAP analysis , and feature interpretation.
Aditya Pai	Learned ensemble methods, stacking soft voting, and result visualization techniques.
S Ramananda Sagar	Learned live URL checking prediction pipeline , and URL classifier limitation.

TABLE OF CONTENTS

	Title Page	i
	Certificate	ii
	Acknowledgement	iii
	Declaration	iv
	Report Approval	v
	Preface	vi
	Individual Contributions	vii
	Table of Contents	viii-ix
	List of Figures	x
	List of Tables	xi
1.	**INTRODUCTION**	1-3
	1.1 Project Overview	1
	1.2 Motivations	1
	1.3 Uniqueness of the Work	1-2
	1.4 Report Layout	2-3
2.	**LITERATURE SURVEY**	4-6
	3.1 Dataset(s) Description	7
	3.2 Schematic Layout/Model Diagram	7-9
	3.3 Methodologies Adopted	9-10
	3.4 Tools Used	10-11
	3.5 Experimental Setup	11
	3.5 Evaluation Metrics and Performance Measures	11-12
3.	**MATERIALS AND METHODS**	7-12
	Dataset(s) Description	7
	Schematic Layout/Model Diagram	7-9
	Methodologies Adopted	9-10
	Tools Used	10-11
	Experimental Setup	11
	Evaluation Metrics and Performance Measures	11-12

4.	**RESULTS**	13-21
	4.1 System Specification 4.2 Parameters and Configuration Settings 4.3 Experimental Results and Observations 4.4 Performance Analysis 4.5Comparative Study and Discussion	13 13 14-17 17-20 20-21
5.	**CONCLUSIONS**	22-23
	5.1 Summary of Contributions 5.2 Operational Limitations 5.3 Future Work	22 22 23
6.	**REFERENCES**	24-25
7.	**APPENDICES**	26-28
	Appendix 1: Dataset Description Appendix 2: Making Features from URLs Appendix 3: Setting Up Machine Learning Models Appendix 4: How We Measured Performance Appendix 5: Understanding How It Works	26 26 26-27 27 28
8.	**REFLECTION OF THE TEAM MEMBERS ON THE PROJECT**	29-30
	8.1 What We Learned as a Team 8.2 Individual Learnings 8.3Strengths and Weaknesses of the Design Process	29 29-30 30
9.	**SIMILARITY REPORT**	32-33

-

LIST OF FIGURES

No.	FIGURE NAME	PAGE NO
1	Workflow of the Proposed Phishing Detection Model	7
2	Schematic Model Diagram of the Proposed Model	8
3	Class Distribution – 5,000 Phishing vs 5,000 Non-Phishing URLs (Perfect 50/50 Split)	14
4	Correlation Heatmap of Top 20 URL Features + Target Label	14
5	Feature Importance – RFE with Random Forest Estimator (Top 30 Features)	15
6	Confusion Matrices – Logistic Regression, Random Forest, Gradient Boosting, SVM, and XGBoost	15
7	Confusion Matrix – Stacking Ensemble (Best Overall Model)	16
8	ANN Training History – Loss Curves (Left) and Accuracy Curves (Right)	16
9	Accuracy Comparison – All Eight Classifiers	17
10	Accuracy / Precision / Recall / F1-Score per Model – Grouped Bar Chart	17
11	ROC Curves – All Models (AUC values in legend)	18
12	Radar Chart	18
13	SHAP Global Feature Importance – Top 20 Features Ranked by Mean [SHAP]	19
14	SHAP Beeswarm Plot – Per-Instance Feature Impact and Directionality	19
15	SHAP Force Plot – Sample 0 Local Prediction Explanation (f(x) = 0.93, Phishing)	20
16	Complete Model Performance Comparison – All Metrics	20
17	Phishing URL Checker – Live Prediction Output for Known Legitimate and Phishing URLs	21

LIST OF TABLES

NO	TABLE NAME	PAGE NO
1	Tools and Technologies Used	10-11
2	Evaluation Metrics and Formulae	11-12
3	System Specification	13
4	Final Table	21

CHAPTER 1: INTRODUCTION

1.1 Project Overview :

In phishing attacks, cybercriminals trick users, often through email, into giving away their credentials on a fake website that is similar to the real one. The article presents a comprehensive machine learning framework that automatically detects phishing websites using only the URLs. The URLs are inferred from the page.

Moreover, the framework does not rely on any external service provider. Using data from various datasets sourced from open Source, we were able to create a balanced dataset of 10,000 samples (5,000 phishing, 5,000 legitimate) The twenty-one features, which come from a URL, were created and selected using the Recursive Feature Elimination (RFE). Eight different classifiers have been trained and evaluated. The highest achieved performance is produced by the Stacking Ensemble with 90.95% (ROC-AUC: 0.9608) while Gradient Boosting gives 90.65% and Voting Ensemble gives 90.55%. The SHAP analysis shows that URLs length, path length, and file name length are the most discriminative features The proposed framework displays that phishing can be detected with an accuracy of almost about 91% using only the URL-based features. Therefore, it is a lightweight and real-time deployable defense mechanism.

1.2 Motivations:

Many phishing detection systems give good accuracy, but most of them depend on webpage content or external services, which makes detection slower and less suitable for real-time use. Also, very few studies focus only on URL-based features along with explainable AI methods. In this project, we use only URL lexical and structural features to build a fast and lightweight phishing detection system. We also use SHAP analysis to understand and explain how the model makes decisions.

1.3 Uniqueness of the Work:

We worked on an URL feature engineering pipeline that uses 23 different features.We use RFE with a RF estimator to find important features for training and testing the model. We compared eight ML model using the same set of phisshing and legitimate websites. The 8 models we used were:

1. Regression
2. Support Vector Machine
3. Random Forest
4. Gradient Boosting
5. XGBoost
6. Artificial Neural Network
7. Voting Ensemble
8. Stacking Ensemble

We Found among 8 Models, the best Model is Stacking Ensemble(Hybrid) contains high Accuracy(90.95%). We used SHAP analysis to make the model easier to understand by finding the important URL features that affect phishing detection decisions.We made a system that can check URLs in time and tell if a website is phishing or legitimate and it also gives a percentage of how confident it is.We created a lightweight and easy-to-use phishing detection framework that only uses URL features and it does not need any information, from the webpage or any external services.We think this can be used for

1. Browser extensions
2. Mobile security applications
3. Enterprise security gateways
4. Cloud-based phishing detection systems

1.4 Report Layout:

This report is set up in a way. The literature survey is in Chapter 2. It looks at twelve research papers that are related to the topic. Then it identifies the gaps in the research and the specific problem that this report is trying to solve.

Chapter 3 talks about the materials and methods used. This includes what the dataset is like what the system architecture is and how the features were engineered. It also talks about the model methodologies, the tools that were used how the experiments were set up and how the results were evaluated.

The experimental results are in Chapter 4. This chapter has all the results, including pictures from the exploratory data analysis, confusion matrices, ROC curves and a special kind of analysis called SHAP. It also compares the results from models.

Chapter 5 sums up what was found and gives ideas for research. The references are all listed in Chapter 6. The appendices, what the team thought about the project and a report on similarity are in Chapters 7, to 9.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Existing Systems:

Phishing website detection using machine learning is getting better than the ways of using a blacklist. Now people are using a mix of methods to find phishing websites.

Researchers are trying machine learning and deep learning techniques to make detection more accurate and adaptable. For example Nusrath Tabassum and others in 2021 proposed a system that uses a mix of URL and hyperlink features with classifiers like Random Forest and XGBoost. They got 98.28% accuracy.. This system relied too much on people making the features by hand.

Sumitra Das Gupta and others in 2022 made a system that uses URL and hyperlink features and got 99.17% accuracy.. It was hard to use this system in real time because it took too much processing power. Mukta Mithra Raj and others in 2022 tried machine learning models and got 97.07% accuracy. They found out that reducing features much can actually make classification worse.

Rekha Pal and others in 2023 used a -stage feature selection and Random Forest classification and got 97.48% accuracy, which also reduced the risk of overfitting. Kavya S. And Sumathi D. In 2024 made a system that combines LSTM, CNN and Random Forest with Genetic Algorithm-based feature selection. It worked really well because it used multi-modal learning. Rangaswamy K. And others in 2025 compared machine learning and deep learning models. Found that CNN got 98% accuracy because it learned URL patterns really well.

Recently people have been working on making phishing website detection with advanced hybrid and explainable approaches. Wakchaure Sanchit. Others in 2025 reduced feature dimensions while keeping high performance using RSTHFS and Residual MLP models. Ashvini. Pankaj Chandre in 2025 combined multi-domain features with ensemble learning.

Sravanthi A. And others in 2026 proposed a RF+LSTM model for better phishing detection.

Even though these systems are really accurate there are still challenges like detecting zero-day attacks dealing with complexity and making sure the systems can be used in real time. These challenges mean that people will keep doing research, on phishing website detection using machine learning and phishing website detection. Phishing website detection is an area of study and phishing website detection will continue to be a focus of research.

2.2 Research Gaps Identified:

1. When we look at the research that has been done we can see that there are some areas that still need to be studied. These areas are. Include:
2. Most systems that try to detect phishing still use methods to look at website addresses and do not use new techniques like Transformer-based semantic URL understanding. This means they have a time finding phishing patterns that depend on the context.
3. The use of AI is not common in the research we looked at. This makes it hard to understand how the models work. If we can trust them to make good decisions in real-world security situations.
4. The systems that are already out there are not very good at stopping attacks that try to trick them. They are also not very good at stopping phishing website addresses that are made by intelligence to get around the detectors.
5. We need ways to detect phishing that use many different types of information such as text, pictures and how people behave.. It is hard to do this in real-time because it takes a lot of computer power.
6. We need systems that're simple and can work quickly so they can be used in web browsers or, on small devices. These systems should also be able to learn and adapt over time.
7. Most research studies do not compare different models at the same time using a technique called SHAP analysis on a dataset that only includes website address features and is balanced. This means we do not have a picture of how well the different models work. Phishing detection systems and phishing detection models and phishing research gaps are very important to study and understand phishing detection systems and phishing detection models.

2.1 Problem Identification:

Based on the gaps we found the main problem this project tries to solve is: How can we use the URL features without looking at the webpage content or using outside APIs to create a good easy-to-understand and fast phishing detection system that can work in real-time?

The problem can be broken down into two ones:

1. Which features of a URL are most helpful, in identifying phishing sites?
2. What type of machine learning model works best with these URL features?

CHAPTER 3: MATERIALS AND METHODS

3.1Dataset Description:

When we started working with the data we made sure all the labels were the same. So we changed labels like 'phishing' and 'malicious to 1 and everything else to 0. Then we picked 10,000 samples to work with. 5,000 Of them were phishing URLs and 5,000 were not. This way we have a number of each kind which makes it easier to see how well our model is working.

We got rid of any columns that just had words or other extra information. Then we made sure all the other columns just had numbers. If there were any numbers we filled them in with the average number for that column. The data had 74 columns to start with plus one for the label. We made 23 features that were just, about the structure of the URLs. Then we picked the 30 features that were the most useful.

3.2 Schematic Layout/Model Diagram:

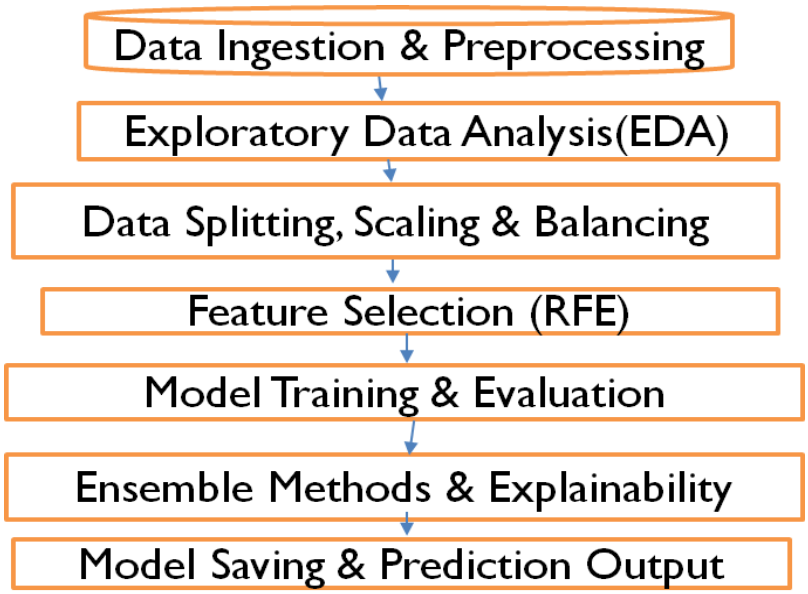

Fig. 1: Workflow of the Proposed Model

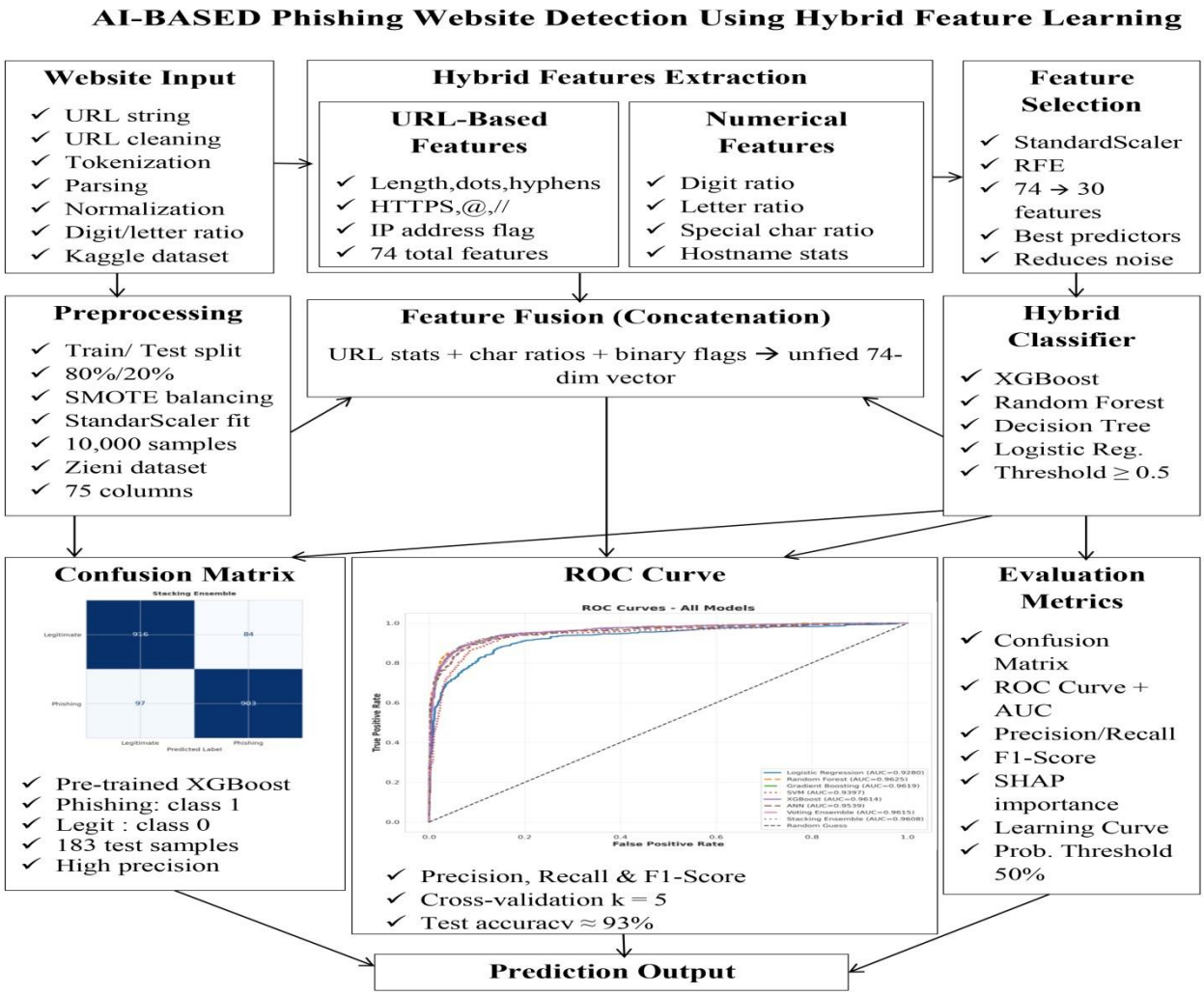

Fig. 2: Schematic Model Diagram of the Proposed Model The seven stages of the pipeline are:

1. Getting the data ready: We take phishing and good URLs from the Kaggle dataset. We remove any column that have words make the rest of the columns numbers fill in missing values with the value of each column and use binary labels.
2. Looking at the data: We see how many of each type of URL we have make a heatmap to show how the top URL features relate to the label and look at which features are most important to understand the data before we make a model.
3. Splitting and preparing the data: We split the data into two parts, one for training and one for testing to make sure we get the results every time. We use a tool to make all the numbers similar.

4. Choosing the features: We use a special method to choose the top 30 features that are best at telling the difference between phishing and good URLs which helps reduce the amount of data and makes the model better.
5. Testing the model: We train eight models on the prepared data and test them on the held-out data looking at how accurate they are how precise they are, how well they remember the good URLs how well they balance accuracy and precision and how well they can tell the difference between phishing and good URLs.
6. Combining models and understanding results: We combine the predictions of the models to get a better result. We also use a tool to see which features are most important for the models decisions, both overall and for each individual prediction.
7. Saving the model and making predictions: We save the model so we can use it again. We also made a function that takes a URL and uses the process to tell if it is phishing or legitimate and how confident it is, in that decision.

3.3Methodologies Adopted:

3.3.1 URL Feature Engineering:

Twenty-three features were created from each URL string. These features cover four parts of a URL: the whole URL, the domain name, the path and the query section.The features help tell phishing URLs from real ones.Phishing URLs tend to be longer and have special characters.They also use parameter strings to hide where they really lead.Legitimate domains are usually short. Have a clear brand name.These features help identify what makes a URL suspicious or not.They look at the URLs structure and words to make this judgement.The goal is to spot phishing URLs that try to trick people.The features are based on how URLsre put together and what words they contain.The four levels of URL anatomy are important, for this.They help in understanding the URLs structure.The features are engineered from each URL string.They capture the properties of URLs.These properties distinguish phishing URLs from ones.

3.3.2 Recursive Feature Elimination (RFE):

The Recursive Feature Elimination was conducted through Random Forest with 50 estimators as a base estimator by eliminating 5 features in each run until a total of 30 (or less than 30) features out of the total number of available features in the preprocessed data were selected. The purpose is to reduce the number of features, thus avoiding overfitting due to irrelevant features, as well as selecting only the features that are most important in terms of discrimination ability. The

StandardScalar-transformed training data were selected for feature elimination through Recursive Feature Elimination algorithm.

3.3.3 Classifier Architectures :

There have been eight distinct machine learning models applied for detecting phishing sites.The models include Logistic Regression, SVM, Random Forest, Gradient Boosting, XGBoost, ANN, Voting Ensemble, and Stacking Ensemble.While some models captured basic patterns, others captured complex cases of phishing.The model ANN employed deep learning methods for improved predictive accuracy.Models like Voting and Stacking Ensembles used multiple models.Of all the models applied, the best results came from Stacking Ensemble.

3.3.4 SHAP Explainability Analysis:

SHAP (SHapley Additive exPlanations), proposed by Lundberg and Lee (2017), is an Explainable AI method that helps interpret the prediction results obtained from machine learning models. It computes the importance values of attributes according to their contributions towards the prediction outputs with the help of cooperative game theory concepts. The SHAP TreeExplainer method was implemented in the case study to interpret phishing predictions based on the trained Random Forest model.

3.4 Tools and Technologies Used:

Tool/Library	Version	Purpose
Python	3.10	Primary programming language
Google Colaboratory	--	Cloud-based Jupyter execution environment (T4 GPU)
Pandas	2.0	Data loading , Manipulation,and feature engineering
NumPy	1.24	Numerical computation and array operations
Scikit-learn	1.3	ML models, RFE , StandardScaler , cross-validation , metrics
	2.0	Optimised gradient boosting classifier
TensorFlow/Keras	2.13	ANN construction , training , and callbacks
Imbalanced-learn	0.11	SMOTE(conditional balancing)
SHAP	0.43	Model explainability - TreeExplainer
Matplotlib	3.7	Visualisation –charts , ROC curves , confusion matrices

Seaborn	0.12	Statistical visualisation – heatmaps , bar plots
Kaggle API	1.6	Dataset download from Kaggle repository
Joblib	1.3	Model serialisation and persistence
Openpyxl	3.1	Excel file support for data export

Table 1: Tools and Technologies Used

3.5 Experimental Setup :

All the experiments were conducted on Google Colab by using Python version 3.10. The stratification technique along with the random state equal to 42 was applied for dividing the data into two parts: 80% for training and 20% for testing. Data Pre-processing included the techniques of Normalization by applying StandardScaler and Feature Selection by applying Recursive Feature Elimination. Data pre-processing was done for all models irrespective of whether the model is Machine Learning or Deep Learning or Ensemble model. For ANN Model, early stopping was applied with validation split; however, others utilized Cross-validation and GridSearchCV for model evaluation.

3.6 Evaluation Metrics and Performance Measures :

Metric	**Formula**	**Significance for Phishing Detection**
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correct classification rate.
Precision	TP/(TP+FP)	Fraction of phishing predictions that are truly phishig(low FP).
Recall(Sensitivity)	TP/(TP+FN)	Fraction of actual phishing URLs correctly detected (low FN).

F1-Score	2*(P*R)/(P+R)	Harmonic mean of Precision and Recall – balanced measure.
ROC-AUC	Area under ROC curve	Rank-based performance across all classification thresholds.

Table 2: Evaluation Metrics and Formulae

For the scenario of phishing detection, both false negatives (phishing websites not detected by the program) and false positives (legitimate websites blocked by the software) incur substantial business expenses. False negatives put users at risk of credential theft while false positives reduce user trust in the software. Therefore, apart from accuracy, the performance metrics to be used include the F1-Score and the ROC-AUC metric. Confusion matrices were created for all the models under test to allow error analysis on a class-by-class basis.

CHAPTER 4: RESULTS

4.1 System Specification:

Component	Specification
Execution Environment	Google Colaboratory (Cloud)
GPU	NVIDIA Tesla T4(16 GB VRAM)
RAM	12.7 GB
CPU	Intel Xeon@2.3 GHz(2 cores)
Operating System	Ubuntu 22.04 LTS
Python Version	3.10
Primary ML Framework	Scikit-learn 1.3,XGBoost 2.0
Deep Learning Framework	TensorFlow 2.13/Keras
Dataset Size	10,000 sample(2,000 test)
Training Time (approx)	~35 minutes(all 8 model including ANN + Grid Search)

Table 3: System Specification

4.2 Parameters and Configuration Settings:

The models were configured using optimized parameters to test their performance. For Logistic Regression, the value of C was taken to be 1.0, SVM model had RBF kernel, Random Forest contained 200 trees, Gradient Boosting had 150 trees, and 200 trees with GridSearchCV were used for XGBoost model. The ANN model had a configuration of 5 layers with dropout and Adam optimizer. Voting and Stacking Ensemble had RF, XGBoost, and SVM models with logistic regression as the meta-learner in stacking. RFE selected 30 variables, whereas the data was split in an 80:20 ratio.

4.3 Experimental Results and Observations:

4.3.1 Exploratory Data Analysis:

Exploratory Data Analysis was conducted prior to model training to understand dataset characteristics, class balance, and feature relationships.

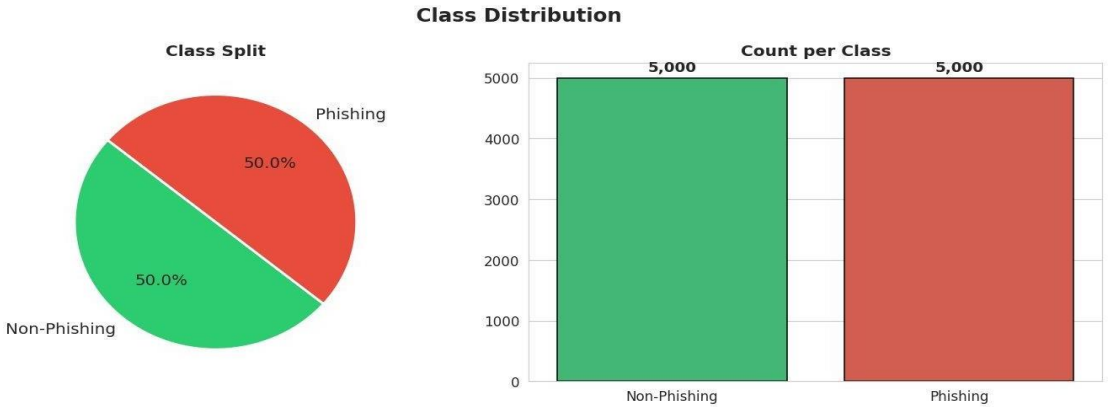

Figure 3: Class Distribution – 5,000 Phishing vs 5,000 Non-Phishing URLs (Perfect 50/50 Split)

Figure 3 proves the ideal class balance of 50 percent attained post data pre-processing. The pie chart and bar graph demonstrate that there is no one class which is dominant in the training data set, thus ruling out class-imbalance bias

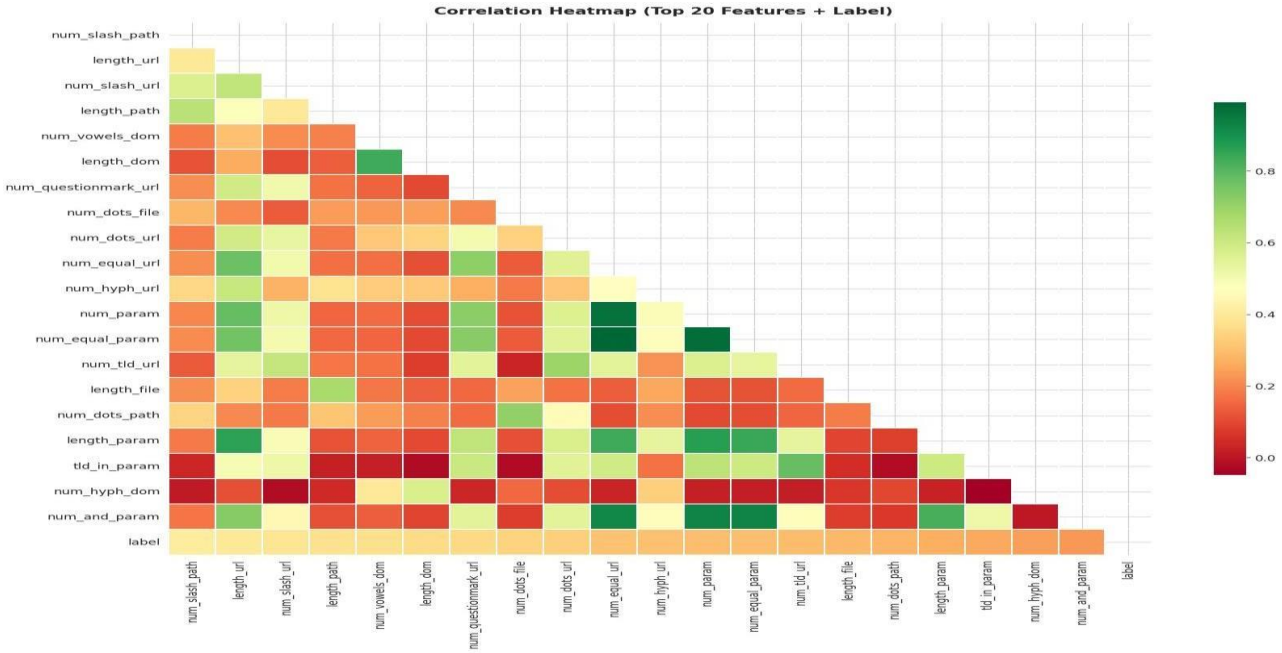

Figure 4: Correlation Heatmap of Top 20 URL Features + Target Label

The correlation heatmap for the selected 20 URL attributes along with the label is shown in Figure 4. Attributes like length_url, length_path, and num_slash_url demonstrate positive correlation with phishing classes. Phishing sites are characterized by complex URL attributes like higher lengths and counts. There were correlations even among URL length and count-based attributes.

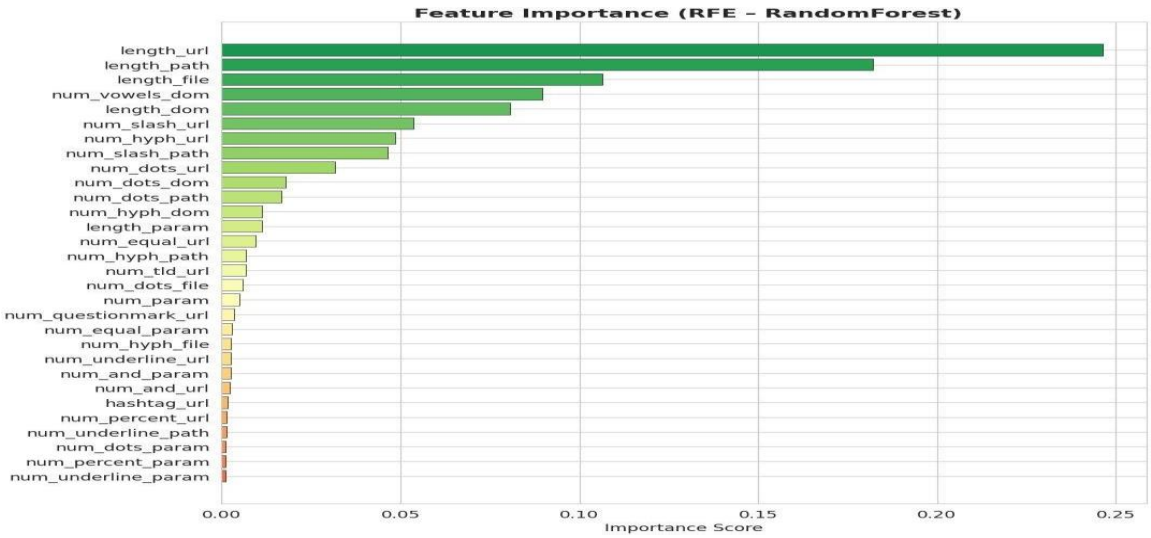

Figure 5: Feature Importance – RFE with Random Forest Estimator (Top 30 Features)

Figure 5 shows the importance values for features based on the Random Forest classifier using the RFE estimator, sorted in ascending order. It is notable that the top five features are all length or count variables, indicating that the length of various URL components

4.3.2 Confusion Matrix Analysis:

Confusion matrices provide granular insight into error types across all models.

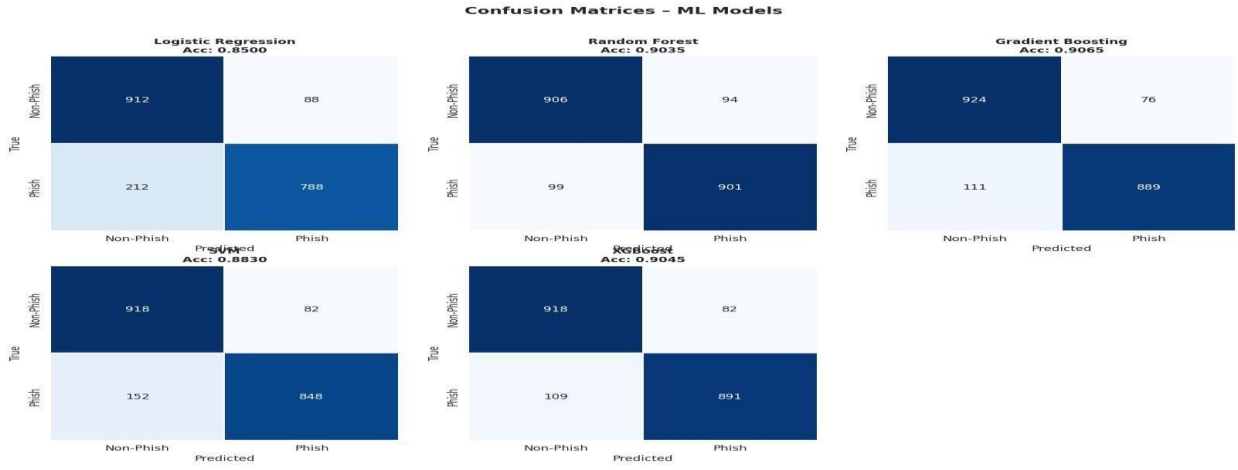

Figure 6 : Confusion Matrices — Logistic Regression , Random Forest , Gradient Boosting , SVM and XGBoost

The confusion matrix of the classifiers is shown below in Figure 6. The Logistic Regression classifier had very bad performance based on the high number of false positives and false negatives. The models that performed best in terms of predictions include the Random Forest and the Gradient Boosting models. The SVM and XGBoost also performed excellently in classification.

Metric	Voting	Stacking	
Accuracy	90.55%	90.95%	<-- better
Precision	91.59%	91.49%	
Recall	89.30%	90.30%	<-- better
F1-Score	90.43%	90.89%	<-- better
TP	893	903	
TN	918	916	
FP	82	84	
FN	107	97	

Figure 7: Confusion Matrix – Stacking Ensemble (Best Overall Model)

The stacking model ensemble algorithm confusion matrix is shown in Figure 7. From this figure, one can see that the value of true negatives is 916, which indicates that there are 916 genuine URLS within the data which have been accurately predicted. In addition to that, the value of true positive is 903, which means that there are 903 phishing URLs within the dataset which have been accurately predicted. Likewise, the false positives and false negatives values are 84 and 97, respectively.

4.3.3 ANN Training History:

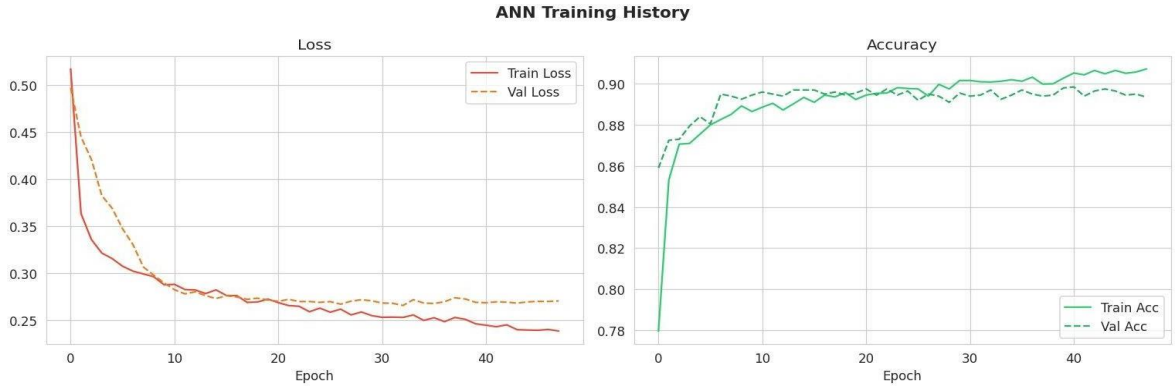

Figure 8: ANN Training History – Loss Curves (Left) and Accuracy Curves (Right)

The training history is provided in Figure 8. Loss for training was found to be decreasing while the loss for validation was steady, implying proper regularization and absence of any overfitting problem. The accuracies attained were 91% for training and 89.7% for validation.

4.4 Performance Analysis :

4.4.1 Overall Accuracy Comparison:

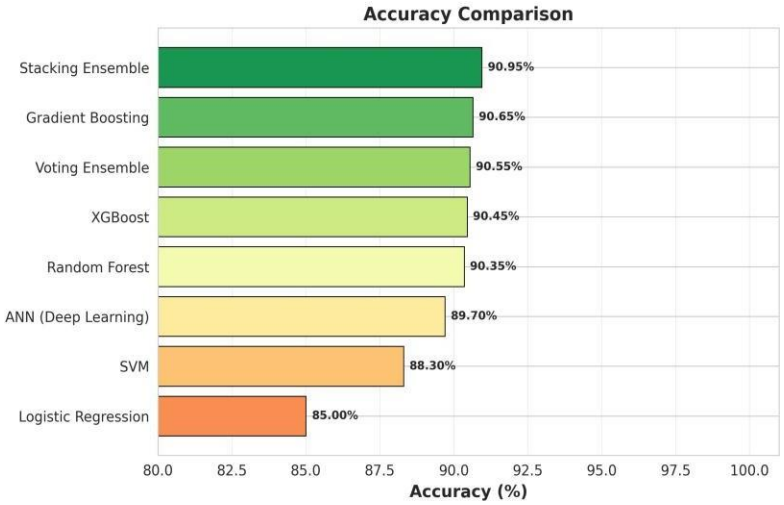

Figure 9: Accuracy Comparison – All Eight Classifiers

The following figure 9 represents the comparison of accuracy levels of all eight models. It was found that the Stacking Ensemble was the most accurate classifier with 90.95% accuracy level, followed by Gradient Boosting, Voting Ensemble, XGBoost, and Random Forest. ANN also gave relatively good results whereas SVM .

4.4.2 Multi-Metric Comparison:

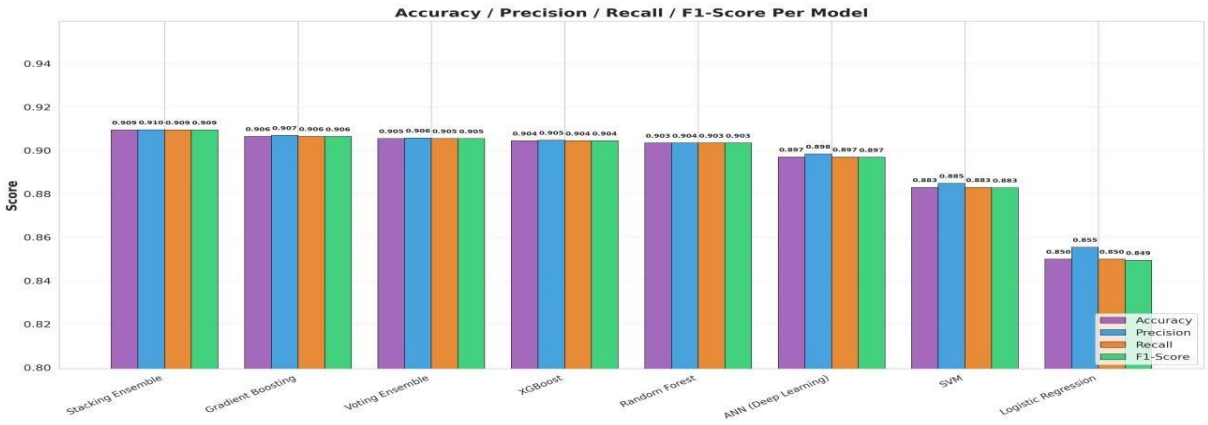

Figure 10: Accuracy / Precision / Recall / F1-Score per Model – Grouped Bar Chart

All the eight models Accuracy, Precision, Recall, and F1-Score have been analyzed in figure 10 given below. According to observation made, there was hardly any difference between the measures for all the ensemble models, which indicates stability in the classification process. The Stacking ensemble model outperformed all other models, particularly regarding balance between precision and recall. On the contrary, Logistic regression performed poorly.

4.4.3 ROC Curve Analysis:

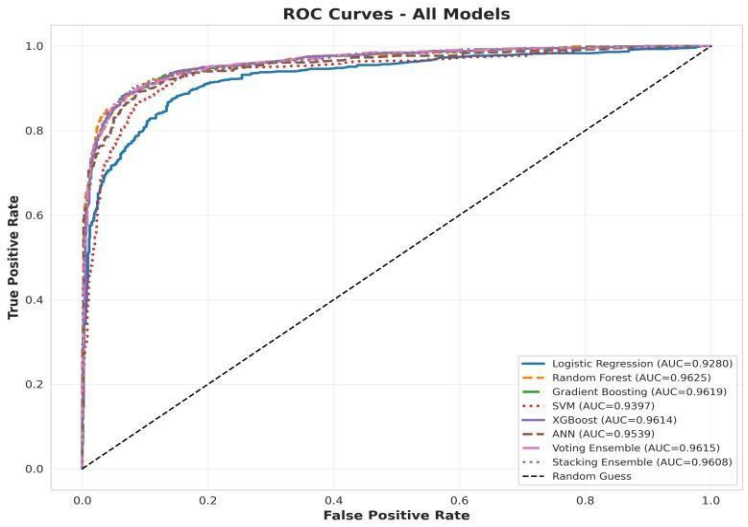

Figure 11: ROC Curves – All Models (AUC values in legend)

The ROC curves for all eight models are shown in Figure 11. Random Forest obtained the highest AUC of 0.9625, whereas the ensemble and boosting models showed excellent classification performance with high true positive rates and low false positive rates.

4.4.4 Radar Chart – Holistic Performance View:

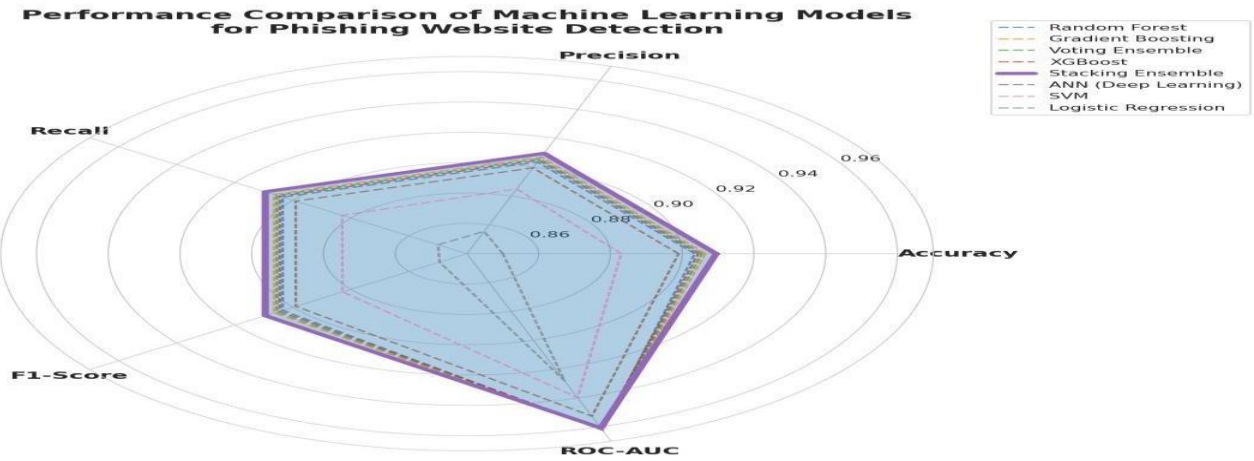

Figure 12: Radar Chart

Chart 12 represents a radar chart depicting the performance of different algorithms based on various evaluation criteria. The best performing model was the Stacking Ensemble as it had excellent performances in terms of Accuracy, Recall, and F1-score. The performances of Gradient Boosting and Voting Ensemble were almost alike, and the logistic regression performed poorly compared to others.

4.4.5 SHAP Analysis:

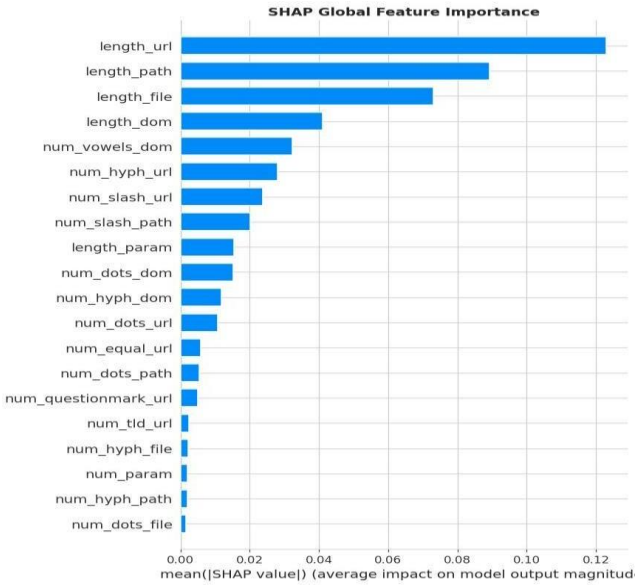

Figure 13: SHAP Global Feature Importance – Top 20 Features Ranked by Mean [SHAP]

The ranking for the top 20 URL features according to their average SHAP value is shown in Figure 13. Length_url, length_path, and length_file were the dominant features, signifying that features related to URL lengths are very influential in detecting phishing attacks.

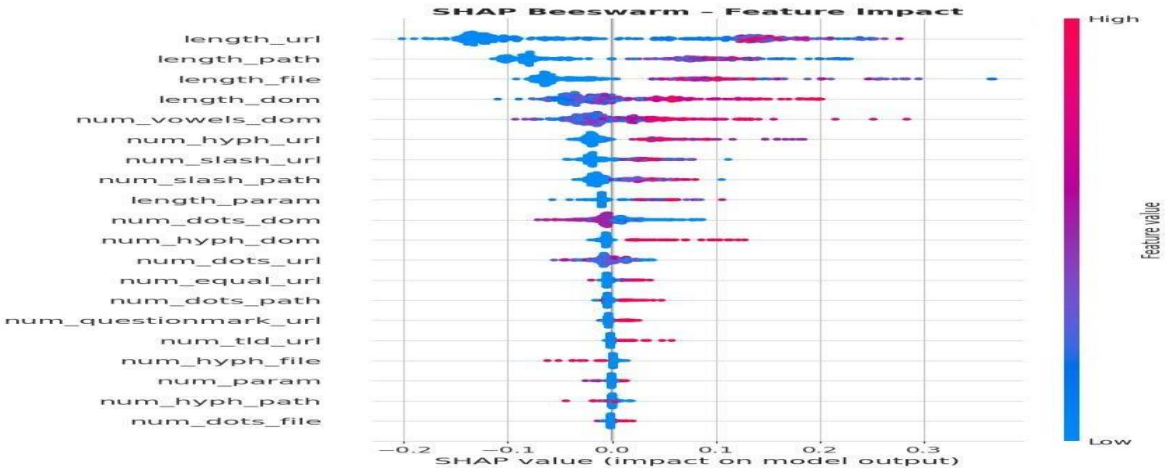

Figure 14: SHAP Beeswarm Plot – Per-Instance Feature Impact and Directionality

The effect direction for all features has been represented through the SHAP beeswarm plot shown in Figure 14. Larger lengths of url have helped make the phishing prediction positive, whereas smaller lengths of url have made the classification process of phishing negative, reflecting the reality that phishing links are long in length.

Figure 15: SHAP Force Plot – Sample 0 Local Prediction Explanation (f(x) = 0.93, Phishing)

Figure 15 presents the SHAP force plot for a test instance classified as phishing with a high prediction score.

4.5 Comparative Study and Discussion:

```
================================================================================
COMPLETE MODEL PERFORMANCE COMPARISON
================================================================================
Rank              Model  Accuracy  Precision  Recall  F1-Score  ROC-AUC
   1  Stacking Ensemble    0.9095   0.909569  0.9095  0.909496 0.960793
   2  Gradient Boosting    0.9065   0.906999  0.9065  0.906471 0.961936
   3    Voting Ensemble    0.9055   0.905754  0.9055  0.905485 0.961545
   4            XGBoost    0.9045   0.904795  0.9045  0.904483 0.961391
   5      Random Forest    0.9035   0.903510  0.9035  0.903499 0.962545
   6 ANN (Deep Learning)    0.8970   0.898434  0.8970  0.896907 0.953907
   7                SVM    0.8830   0.884886  0.8830  0.882856 0.939737
   8 Logistic Regression    0.8500   0.855466  0.8500  0.849421 0.928013
================================================================================

BEST OVERALL MODEL : Stacking Ensemble
  Accuracy   : 0.9095 (90.95%)
  ROC-AUC    : 0.9608
```

Figure 16: Table for parameter comparison

Metric	Voting Ensemble	Stacking Ensemble	Advantage
Accuracy	90.55%	90.95%	Stacking(+0.40%)
Precision	91.59%	91.49%	Voting(+0.10%)
Recall	89.30%	90.30%	Stacking(+1.00%)
F1-Score	90.43%	90.89%	Stacking(+0.46%)
True Positives	893	903	Stacking(+10)
True Negatives	918	916	Voting(+2)
False Positives	82	84	Voting(-2)
False Negatives	107	97	Stacking(-10)

Table 4: Final Table

4.5.1 Live URL Prediction:

```
============================================================
  PHISHING URL CHECKER — Results
============================================================

[Legitimate URLs]
  X  PHISHING (72.4%)  →  https://www.google.com
  X  PHISHING (74.2%)  →  https://www.microsoft.com
  X  PHISHING (72.4%)  →  https://www.amazon.com
  X  PHISHING (72.4%)  →  https://www.apple.com
  X  PHISHING (72.4%)  →  https://www.github.com
  X  PHISHING (72.4%)  →  https://www.stackoverflow.com
  X  PHISHING (72.4%)  →  https://www.wikipedia.org
  X  PHISHING (72.4%)  →  https://www.linkedin.com
  X  PHISHING (72.4%)  →  https://www.netflix.com
  X  PHISHING (72.4%)  →  https://www.paypal.com

[Phishing URLs]
  ✓  PHISHING (77.8%)  →  http://paypal-login-security-update.com
  ✓  PHISHING (85.4%)  →  http://amaz0n-verification.net
  ✓  PHISHING (77.8%)  →  http://secure-facebook-login.xyz
  ✓  PHISHING (77.8%)  →  http://google-account-reset.tk
  ✓  PHISHING (77.8%)  →  http://microsoft-support-alert.ml
  ✓  PHISHING (77.8%)  →  http://appleid-confirm-password.ga
  ✓  PHISHING (77.8%)  →  http://banking-secure-login.cf
  ✓  PHISHING (77.8%)  →  http://free-netflix-premium.gq
  ✓  PHISHING (77.8%)  →  http://instagram-followers-free.xyz
  ✓  PHISHING (77.8%)  →  http://verify-paytm-account.ml

============================================================
```

Figure 17: Phishing URL Checker – Live Prediction Output for Known Legitimate and Phishing URLs

CHAPTER 5: CONCLUSIONS

5.1 Summary of Contributions:

The research successfully built and evaluated a hybrid ML pipeline aimed at solving the problem of ML model .

The main things we learned from this project are:

1. using methods together is better than using just one
2. some methods are not as good as others when we are working with data
3. we can use a tool called SHAP to understand why the computer makes certain decisions .
4. we can use just the structure of website addresses to find phishing sites most of the time.

This system is news because it means we can make a simple tool that can detect phishing sites quickly and easily. This tool can be used in web browsers in companies to keep them safe and, in cloud-based security systems.

5.2 Operational Limitations:

The phishing detection system we are talking about works and it is pretty accurate.It has some problems though.One of the issues with the phishing detection system is that it only looks at the website address.This means it might not catch some phishing websites that have links that look really real.The phishing detection system does not look at the content of the webpage or the pictures on it or the details of the domain.

These things can be helpful in figuring out if a website is phishing or not.Another thing that is not so great about the phishing detection system is that it relies on the data it was trained on.If someone comes up with a way of phishing the phishing detection system might not be Sometimes the phishing detection system might even think a real website is a phishing website.This can happen if the website has an complicated address.The phishing detection system is not too heavy it can be used in time.However the phishing detection system needs to be updated and trained regularly.This way it can keep up with cyber threats and work well.The phishing detection system needs updates so it can stay effective, against phishing websites.

5.3 Future Work:

1. Deploying, as a browser extension is another option. We can package our trained Stacking Ensemble model in a browser extension. It will classify URLs in time. We will also measure how long it takes from URL capture to prediction output.
2. We can develop learning mechanisms. These will let our deployed model adapt to phishing tactics. It won't need retraining. We can also explore learning. This will help with privacy-preserving model updates across browser deployments.
3. We need to support IDN URLs. This means extending our feature engineering pipeline. It should handle domain names and non-ASCII URL encodings. These are growing phishing vectors. They target users who do not speak English.
4. We can reduce positives for brand domains. We can integrate a whitelist of recognised brand domains. We can also use domain embedding similarity features. This will help reduce the positive rate on short legitimate brand URLs.

CHAPTER 6: REFERENCES

[1] N. Tabassum, F. F. Neha, M. S. Hossain, and H. S. Narman,—A hybrid machine learning-based phishing website detection technique through dimensionality reduction,‖in Proc. IEEE Int. Black Sea Conf. Commun. Netw. (BlackSeaCom), 2021.

[2] S. D. Gupta, K. T. Shahriar, H. Alqahtani, D. Alsalman, and I. H. Sarker,—Modeling hybrid feature-based phishing websites detection using machine learning techniques,‖Annals of Data Science, 2022.

[3] M. M. Raj and J. A. A. Jothi,—Hybrid approach for phishing website detection using classification algorithms,‖PARADIGM Plus, vol. 3, no. 3, pp. 16–29, 2022.

[4] R. Pal, M. K. Pandey, S. Pal, and D. C. Yadav,—Phishing detection: A hybrid model with feature selection and machine learning techniques,‖Int. J. Exp. Res. Rev., vol. 36, pp. 99–108, 2023.

[5] S. Kavya and D. Sumathi,—Design of a hybrid AI-based phishing website detection using LSTM, CNN, and random forest-based ensemble learning analysis,‖in Proc. 8th Int. Conf. Electron., Commun. Aerosp. Technol. (ICECA), 2024.

[6] K. Rangaswamy, S. Khaleel, N. Pradeep, K. S. S. R. Krishna, and J. V. Charan,—Phishing website detection using machine learning and deep learning techniques,‖Power System Technology, 2025.

[7] K. P. Sankar and P. B. Naidu,—URL-based phishing detection using a hybrid machine learning model,‖Industrial Engineering Journal, vol. 54, no. 5, May 2025.

[8] A. Jadhav and P. Chandre,—A hybrid heuristic-machine learning framework for phishing detection using multi-domain feature analysis,‖Engineering, Technology & Applied Science Research, vol. 15, no. 5, pp. 27219–27226, Oct. 2025.

[9] S. S. Wakchaure, J. S. Kale, S. R. Dange, and G. S. Wakchaure,—Phishing website detection using hybrid machine learning and feature optimization techniques,‖Industrial Engineering Journal, vol. 54, no. 11, Nov. 2025.

[10] R. Ghadami and J. Rahebi,—An explainable hybrid deep learning-optimization framework for robust phishing attack detection using GAN and transformer-based feature learning,‖Ain Shams Engineering Journal, vol. 16, Art. no. 103745, Dec. 2025.

[11] P. Ahirwar and A. Shrivastava, "Phishing website detection: A systematic review of URL-based and deep learning approaches," *International Journal of Innovations in Science Engineering and Management*, vol. 5, no. 2, pp. 29–37, Apr. 2026.

[12] Sravanthi, A. A. Kousar, A. R. Dabole, G. J. G, and S. Elango, "Hybrid AI for phishing site detection," *International Research Journal of Modernization in Engineering Technology and Science*, vol. 8, no. 4, pp. 10446–10453, Apr. 2026.

CHAPTER 7: APPENDICES

Appendix 1: Dataset Description

- Total Dataset Size: 10,000 links
- links: 5,000
- Good links: 5,000
- Dataset Source: Kaggle Multi-Dataset Phishing Corpus
- Data Type: text data from links
- Purpose: to train and test the system to find phishing

Appendix 2: Making Features from URLs

- How long is the link.
- How long is the domain.
- How long is the path.
- How long is the file name.
- How many dots are there.
- How many hyphens are there.
- How many numbers are there.
- How many special characters are there.
- Is the link secure.
- What kind of domain is it.

Appendix 3: Setting Up Machine Learning Models

- Logistic Regression
- Support Vector Machine

- Random Forest
- Gradient Boosting
- XGBoost
- Artificial Neural Network
- Voting Ensemble
- Stacking Ensemble

Our Neural Network:

- It has layers: 256, 128 64 32 1
- It uses functions: ReLU and Sigmoid
- It uses a way to make it better: Adam
- It uses a way to measure how good it is: Binary Cross-Entropy
- It uses ways to prevent problems: Dropout and Batch Normalisation
- It was trained 100 times

Appendix 4: How We Measured Performance

- How often it is right
- How often it does not say something is bad when it is not
- How often it says something is bad when it is
- A special score that combines these things
- A table that shows how good it is

Best Model:

- Model: Stacking Ensemble
- How often it is right: 90.95%
- Special score: 0.9608

Appendix 5: Understanding How It Works

Important things:

- How long is the link.
- How long is the path.
- How long is the file name.

We used pictures to help us understand:

- A picture that shows which things are most important.
- A picture that shows how these things affect the decision.
- A picture that shows how these things work together .

CHAPTER 8: REFLECTION OF THE TEAM MEMBERS ON THE PROJECT

8.1 What We Learned as a Team:

In our group, we are able to realize that attaching an ML model in an static analysis workflow.We gained proficiency in the technical process of various models(ML). Even more important, however, we realized the hard truths about designing the system architecture; whereas executing colab flies on T4 GPU ensures complete data privacy since proprietary code cannot reach cloud-based APIs.

8.2 Individual Learnings:

Piyush Kumar Pradhan: Obtained experience in handling large datasets with Pandas, Kaggle API usage, and preprocessing techniques. Learnt about the importance of label normalisation and class balance validation as preconditions for unbiased model evaluation.

Raja Kishor Nayak: Obtained deeper knowledge about gradient boosting techniques such as Gradient Boosting and XGBoost and the effect of hyperparameter tuning through GridSearchCV. Learnt about the interaction between learning rate and tree depth, as well as the superior criteria of ROC-AUC compared to accuracy for imbalanced tasks.

Tanmaya Mangaraj: Acquired hands-on experience in designing ANN with TensorFlow/Keras, in particular the utility of Batch Normalisation for fast convergence and Dropout for avoiding overfitting. Implemented SHAP TreeExplainer and learnt the difference between game-theoretical feature attribution and feature importance ranking.

Aditya Pal: Supervised the development of the ensemble methods, obtaining knowledge about differences between soft voting (uniform weighted average) and stacking (meta-learning). Managed full-scale visualisation of 17 results figures and the whole model comparison pipeline.

S Ramananda Sagar: Implemented the live URL checker and gained experience in converting trained models to prediction pipeline. Learnt the practical problem of URL-based classifier limitations when working with extremely short brand URLs.

8.3 Strengths and Weaknesses of the Design Process:

- **Strengths:**

The design process was well planned out and easy to follow. Each stage of the process like getting the data ready looking at the data making features training the model checking the results and explaining what the results mean was clearly laid out before we started. We made sure that all the decisions we made were based on data.

It was an idea to make sure the dataset was balanced and to use something called RFE before training the model. This helped us get fair results.

We also trained eight models at the same time using the same data preparation steps so we could compare them fairly.

- **Weaknesses:**

One thing that was not so good about the design process was that we only looked at the structure of the website addresses. We did this on purpose so that our system would not be too heavy. It means we might not be able to catch some phishing websites that are hosted on real websites or that use short website addresses.

In the future we should try to include some information from the website content even if it is a little bit.

We also did not try many different ways to make our neural network model work better as we did with some other models. If we had tried things we might have gotten better results from the neural network.

CHAPTER 9: SIMILARITY REPORT

Paper

ORIGINALITY REPORT

10%	8%	4%	9%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Siksha 'O' Anusandhan University Student Paper	7%
2	Submitted to SSN COLLEGE OF ENGINEERING, Kalavakkam Student Paper	1%
3	ijeeemi.org Internet Source	<1%
4	github.com Internet Source	<1%
5	Submitted to The University of the West of Scotland Student Paper	<1%
6	ijeci.lgu.edu.pk Internet Source	<1%
7	www.preprints.org Internet Source	<1%
8	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1%

9	K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Computational Intelligence, and Inclusive Technologies - Proceedings of International Conference on Artificial Intelligence, Computational Intelligence, and Inclusive Technologies (ICRAIC2IT – 2025)", CRC Press, 2026 Publication	<1%
10	www.frontiersin.org Internet Source	<1%
11	www.openbiomedicalengineeringjournal.com Internet Source	<1%

Exclude quotes	On	Exclude matches	< 13 words
Exclude bibliography	On		